Day 6

Data Structures

Lists · Tuples · Dictionaries · Sets · Comprehensions · Slicing

Session Length	Phase	Difficulty	Prerequisites
2 hours	Beginner	■■■ Intermediate	Days 1–5

- ✔ Create and manipulate **lists**, **tuples**, **dictionaries**, and **sets**
- ✔ Distinguish mutability: lists/dicts/sets are mutable; tuples are immutable
- ✔ Use slicing syntax to extract sub-sequences from lists and strings
- ✔ Write concise list, dict, and set **comprehensions**
- ✔ Choose the right data structure for a given problem
- ✔ Apply the most common built-in methods for each type

■ SECTION 1 — SESSION OVERVIEW

Session Goal

Introduce Python's four core collection types and teach students to select and use the correct structure for any data-storage problem. By the end they will write comprehensions fluently and manipulate nested data confidently.

Session Outcome

Students leave with a working *contact book* mini-project that stores people's names, phone numbers, and tags — built entirely with dicts and lists.

Minute-by-Minute Timeline

Time	Activity	Type
0:00–0:10	Recap Day 5 functions quiz (3 quick questions)	Review
0:10–0:25	Lists — creation, mutation, methods	Teach + Live Code
0:25–0:35	Tuples — immutability, packing / unpacking	Teach + Live Code
0:35–0:50	Dictionaries — key-value pairs, methods	Teach + Live Code
0:50–1:00	Sets — unique collections, set operations	Teach + Live Code
1:00–1:10	Slicing syntax deep-dive	Teach + Exercises
1:10–1:30	List / dict / set comprehensions	Teach + Live Code
1:30–1:50	Mini-project: Contact Book	Guided Build
1:50–2:00	Recap, Q&A, Day 7 preview	Wrap-Up

■ Instructor Note

Data structures feel abstract until students see a real use-case.
Anchor each type with a memorable analogy:
- List → numbered shopping list (ordered, changeable)
- Tuple → GPS coordinate (fixed once set)
- Dict → phone book (look up by name, not by position)
- Set → highlighter colours on a page (each appears only once)

Return to these analogies whenever students look confused.

■ SECTION 2 — PREREQUISITES & PREPARATION

Student Prerequisites

Concept	Covered On	Why It Matters Today
Variables & data types	Day 2	Values stored in collections have types
Control flow (if/elif/else)	Day 3	Needed for filtering in comprehensions
for loops & range()	Day 4	Iteration over collections
Functions & scope	Day 5	Mini-project uses helper functions

Pre-Session Instructor Checklist

- ■ Open VS Code with a fresh file day06_live.py
- ■ Terminal ready in the project virtualenv (uv run python)
- ■ Projector / screen share showing the REPL clearly
- ■ Have this list ready to paste: fruits = ['apple','banana','cherry','date','elderberry']
- ■ Have this dict ready to paste: person = {'name': 'Priya', 'age': 28, 'city': 'Mumbai'}
- ■ Confirm all students have completed Day 5 exercises

■ SECTION 3 — STEP-BY-STEP INSTRUCTION GUIDE

Step 1 — Lists

What to Say:

"A list is Python's most flexible container. Think of it as a numbered shopping list — ordered, changeable, and you can have duplicates. Lists use square brackets."

Live Code:

```
# Creating lists
fruits = ['apple', 'banana', 'cherry']
numbers = [1, 2, 3, 4, 5]
mixed  = [42, 'hello', True, 3.14]    # any types allowed

# Indexing (zero-based)
print(fruits[0])       # apple
print(fruits[-1])      # cherry  (negative = from end)

# Mutation
fruits.append('date')           # add to end
fruits.insert(1, 'avocado')     # insert at index 1
fruits.remove('banana')         # remove first match
popped = fruits.pop()           # remove & return last

# Useful methods
fruits.sort()                   # sort in-place
print(len(fruits))              # number of items
print('apple' in fruits)        # membership test
```

Common Errors:

Error	Cause	Fix
IndexError: list index out of range	fruits[10] on 3-item list	Check len() before indexing
TypeError: 'str' object does not support item assignment	string[0]='X'	Strings are immutable; use a list

■ **Done When...**

Students can create a list, append/remove items,
index both forward and backward, and use len() correctly.

Step 2 — Tuples

What to Say:

"A tuple is like a list but locked. Once created you can't change it. Use tuples for data that should never change — coordinates, RGB colours, database rows. Parentheses, or just commas."

Live Code:

```
# Creating tuples
point   = (10, 20)          # x, y coordinate
colours = ('red', 'green', 'blue')
single  = (42,)             # trailing comma required for single item!

# Indexing — same as lists
print(point[0])    # 10

# Unpacking
x, y = point
print(f'x={x}, y={y}')   # x=10, y=20

# Tuples are immutable
# point[0] = 99    # TypeError!

# Useful: swap variables
a, b = 1, 2
a, b = b, a      # tuple trick!
print(a, b)      # 2 1
```

Step 3 — Dictionaries

What to Say:

"Dictionaries store key-value pairs. Think of a real phone book — you look up by name (key) to get the number (value). Keys must be unique and immutable. Since Python 3.7 dicts maintain insertion order."

Live Code:

```python
# Creating a dict
person = {
    'name': 'Priya',
    'age':  28,
    'city': 'Mumbai'
}

# Access
print(person['name'])          # Priya
print(person.get('email', 'N/A'))   # safe access with default

# Mutation
person['email'] = 'priya@example.com'   # add new key
person['age']   = 29                    # update existing
del person['city']                      # remove key

# Iteration
for key, value in person.items():
    print(f'{key}: {value}')

# Useful methods
print(person.keys())     # dict_keys([...])
print(person.values())   # dict_values([...])
print('name' in person)  # True
```

■■ Common Mistake

person['missing_key'] raises KeyError.
Always use person.get('key', default) when the key might not exist.

Step 4 — Sets

What to Say:

"A set is an unordered collection with no duplicates. Perfect for membership testing and removing duplicates. Sets use curly braces, but an empty set must be set(), not {} — that's an empty dict!"

Live Code:

```
# Creating sets
tags = {'python', 'web', 'data', 'python'}  # duplicate dropped!
print(tags)    # {'python', 'web', 'data'}

# Add / remove
tags.add('api')
tags.discard('web')   # safe remove (no error if missing)

# Set operations
a = {1, 2, 3, 4}
b = {3, 4, 5, 6}
print(a | b)    # union        {1,2,3,4,5,6}
print(a & b)    # intersection {3,4}
print(a - b)    # difference   {1,2}
print(a ^ b)    # symmetric diff {1,2,5,6}

# Fast membership test  O(1)  vs list O(n)
print('python' in tags)  # True

# Remove duplicates from a list
nums = [1,2,2,3,3,3,4]
unique = list(set(nums))
```

Step 5 — Slicing Syntax

What to Say:

"Slicing lets you grab sub-sequences from any ordered type — lists, tuples, strings. The syntax is sequence[start:stop:step]. Think of start as inclusive and stop as exclusive."

Live Code:

```
nums = [0, 1, 2, 3, 4, 5, 6, 7, 8, 9]

print(nums[2:5])      # [2, 3, 4]   start=2, stop=5 (exclusive)
print(nums[:4])       # [0, 1, 2, 3]  omit start → from beginning
print(nums[6:])       # [6, 7, 8, 9]  omit stop → to end
print(nums[::2])      # [0, 2, 4, 6, 8]  every 2nd element
print(nums[::-1])     # [9,8,...,0]   reverse!
print(nums[-3:])      # [7, 8, 9]    last 3

# Works on strings too
word = 'Python'
print(word[1:4])   # 'yth'
print(word[::-1])  # 'nohtyP'  — quick reverse
```

Step 6 — Comprehensions

What to Say:

"Comprehensions are Python's concise way to build collections in a single readable expression. They replace many simple for-loops. The pattern is always: [expression for item in iterable if condition]."

Live Code:

```
# List comprehension
squares = [x**2 for x in range(10)]
# vs old-style:
# squares = []
# for x in range(10): squares.append(x**2)

# With a filter condition
evens = [x for x in range(20) if x % 2 == 0]
print(evens)  # [0, 2, 4, 6, 8, 10, 12, 14, 16, 18]

# Dict comprehension
words = ['apple', 'banana', 'cherry']
lengths = {word: len(word) for word in words}
print(lengths)  # {'apple': 5, 'banana': 6, 'cherry': 6}

# Set comprehension
unique_lengths = {len(word) for word in words}
print(unique_lengths)  # {5, 6}

# Nested list comprehension
matrix = [[i*j for j in range(1,4)] for i in range(1,4)]
# [[1,2,3],[2,4,6],[3,6,9]]
```

■ **When to use a comprehension vs a loop**

Comprehension → single expression that builds a new collection

Loop → side-effects (printing, writing to DB, appending conditionally to multiple lists)

Rule of thumb: if it fits on one readable line, use a comprehension.

■ SECTION 4 — SESSION WRAP-UP

Student Exit Checklist

- ■ I can create lists, tuples, dicts, and sets from scratch
- ■ I understand which types are mutable and which are immutable
- ■ I can use slicing to extract sub-sequences
- ■ I can write a list comprehension with a filter condition
- ■ I can write a dict comprehension
- ■ I know when to use a set for fast membership testing
- ■ I built the contact book mini-project successfully

Key Concepts Recap

Structure	Syntax	Ordered?	Mutable?	Duplicates?
list	[1, 2, 3]	■ Yes	■ Yes	■ Yes
tuple	(1, 2, 3)	■ Yes	■ No	■ Yes
dict	{'k': v}	■ Yes*	■ Yes	Keys: ■
set	{1, 2, 3}	■ No	■ Yes	■ No

* Insertion order preserved since Python 3.7

Day 7 Preview — Strings & String Methods

Tomorrow we go deeper into strings: f-strings, formatting, split/join, strip/replace, and the re module for pattern matching. Encourage students to look at Python's str documentation tonight.

■ Troubleshooting Notes

KeyError in dict → student used [] on missing key. Teach .get()
Set appears 'out of order' → expected; sets are unordered by design
Comprehension confusion → break it into a loop first, then collapse
Tuple with one element → remind about trailing comma: (x,)
Mutating while iterating → common mistake; iterate a copy: list(d.keys())

■ SECTION 5 — MINI-PROJECT: CONTACT BOOK

Walk students through building a simple contact book CLI. This ties together dicts (storing contacts), lists (multiple phone numbers), sets (tags), and comprehensions (searching).

Guided Build Steps:

1. Define contacts as a dict of dicts: {name: {'phone': ..., 'tags': set()}}
2. Write add_contact(contacts, name, phone, tags) function
3. Write search_by_tag(contacts, tag) using a list comprehension
4. Write display_contact(contacts, name) with formatted output
5. Add a simple menu loop: add / search / view / quit

```
# Starter scaffold
contacts = {}

def add_contact(contacts, name, phone, tags):
    contacts[name] = {
        'phone': phone,
        'tags':  set(tags)   # convert list to set
    }

def search_by_tag(contacts, tag):
    return [name for name, info in contacts.items()
            if tag in info['tags']]

# Usage
add_contact(contacts, 'Priya',   '+91-9000000001', ['friend','work'])
add_contact(contacts, 'Arjun',   '+91-9000000002', ['family'])
add_contact(contacts, 'Lakshmi', '+91-9000000003', ['friend'])
print(search_by_tag(contacts, 'friend'))  # ['Priya', 'Lakshmi']
```

■ SECTION 6 — QUICK REFERENCE CARD

Essential Methods

Type	Method / Syntax	What It Does
list	lst.append(x)	Add x to end
list	lst.insert(i, x)	Insert x at index i
list	lst.remove(x)	Remove first occurrence of x
list	lst.pop()	Remove & return last item
list	lst.sort() / sorted(lst)	Sort in-place / return new sorted
list	lst[start:stop:step]	Slice
dict	d.get(key, default)	Safe access with fallback
dict	d.items() / .keys() / .values()	Iterate pairs / keys / values
dict	d.update(other)	Merge another dict in
set	s.add(x) / s.discard(x)	Add / safely remove
set	s | t, s & t, s - t, s ^ t	Union/Intersection/Diff/Sym.diff
all	len(x) / x in y	Length / membership test

Comprehension Cheat-Sheet

```
# List comprehension
[expr  for item in iterable]
[expr  for item in iterable  if condition]

# Dict comprehension
{key_expr: val_expr  for item in iterable}

# Set comprehension
{expr  for item in iterable}

# Slicing quick-ref
seq[start:stop]   # items from start up to (not including) stop
seq[::step]       # every nth item
seq[::-1]         # reverse any sequence
```

Choosing the Right Structure

If you need...	Use
Ordered, changeable collection	list

Fixed, immutable sequence	tuple
Multiple values returned from a function	tuple
Key→value lookup	dict
Unique items / fast membership test	set
Eliminate duplicates from a list	set(list) → list